



Universidad
Continental

INGENIERÍA WEB

Mtr. Jisbaj Gamarra Salas

Guía de laboratorio 02

Tema: Fundamentos de HTML y CSS

Ingeniería Web

Propósito de la sesión

Al **finalizar** esta práctica podrás:

- reconocer la función de HTML y CSS dentro de una página web;
- distinguir elementos **block**, **inline** e **inline-block**;
- aplicar estilos básicos con selectores, clases e identificadores;
- construir pequeños ejemplos visuales entendiendo por qué se comportan de cierta manera.

Recordatorio: ¿qué es HTML?

HTML es el lenguaje que describe el **contenido** y la **estructura** de una página web.

Con HTML indicamos:

- títulos;
- párrafos;
- enlaces;
- imágenes;
- contenedores de contenido.

No se usa para programar lógica, sino para **organizar información**.

Estructura básica de una página HTML

Plantilla base para copiar y pegar:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Ingeniería web</title>
  </head>

  <body>
    ... contenido de la página ...
  </body>
</html>
```

Se guarda en un archivo con extensión **.html**.

Estructura básica de una página HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>Ingeniería web</title>
  </head>

  <body>
    ... contenido de la página ...
  </body>
</html>
```

- **<head>** : metadatos que no aparecen en el área visible del navegador.
- **<body>** : contenido que sí se renderiza en la ventana del navegador.
- Ejemplo: **<title>** aparece como el nombre de la pestaña.

Elementos HTML

Un elemento HTML normalmente:

- Tiene etiqueta de apertura y cierre, por ejemplo: `<p>` y `</p>`.
- Tiene **contenido** entre ambas etiquetas.
- Puede ser **autocerrado**, como `<img />`.
- Puede tener **atributos**, por ejemplo `src="puppy.jpg"`.
- Puede contener otros elementos.

```
<p>  
  HTML es <em>genial</em>  
    
</p>
```

Algunos elementos HTML frecuentes

Elementos de estructura y contenido:

- `h1` , `h2` , `h3` ... `h6`
- `p`
- `a`
- `img`
- `strong`
- `em`
- `br`

Cada uno tiene un propósito semántico. Elegir la etiqueta correcta ayuda a que el documento sea más claro y mantenible.

Algunos elementos HTML

Elementos comunes para usar dentro de `<body>` :

- Encabezado principal: `h1` , `h2` , ... `h6`
 - `<h1>Moby Dick</h1>`
- Párrafo:
 - `<p>Call me Ishmael.</p>`
- Salto de línea:
 - `since feeling is first<br/>who pays any attention`
- Imagen:
 - ``

- Enlace:

- `<a href="google.com">haz clic aquí</a>`

- Texto en negrita:

- `<strong>Con fuerza</strong>`

- Énfasis o cursiva:

- `<em>Es mi hermano</em>`

Elementos HTML autocerrados

- No contienen texto ni otros elementos en su interior, por lo que no requieren una etiqueta de cierre (ej. `<img>`, `<br>`).
- Se caracterizan por ser autosuficientes y definen elementos insertados directamente.
 - br: Inserta un salto de línea.
 - img: Incrusta una imagen.
 - input: Define un campo de entrada de datos.
 - meta: Define metadatos del documento (charset, viewport).
 - link: Enlaza recursos externos (ej. CSS).
 - hr: Inserta una regla horizontal (línea temática).
 - col: Define atributos de columna en una tabla.

Página del curso

```
<!DOCTYPE html>
<html>
  <head>
    <title>Ingeniería web</title>
  </head>
  <body>
    <h1>Web Fun</h1>
    <strong>Anuncios:</strong><br />
    Tarea 1 vencida!<br />
    Elegir bien el tema de proyectp<br />
    <a href="http://www.netflix.com"> Ver inspiración </a>
  </body>
</html>
```

¿Qué observamos?

Eso fue extraño

Observamos que:

- Los espacios en HTML colapsan en **un solo espacio**.
- Pero, curiosamente, el encabezado `<h1>` aparece en una línea propia y `<strong>` no.

Más adelante entenderemos por qué ocurre esto.

Recordatorio:

¿Qué es CSS?

CSS: Cascading Style Sheets (Hojas de Estilo en Cascada)

- CSS describe la **apariciencia** y parte de la **disposición visual** de una página.
- Está compuesto por **reglas CSS**, que definen conjuntos de estilos.

```
selector {  
  property: value;  
}
```

CSS

Una regla CSS tiene esta forma:

```
selector {  
  property: value;  
}
```

- **selector**: especifica qué elemento(s) HTML se van a estilizar.
- **property**: el nombre de la propiedad CSS.
- **value**: el valor de esa propiedad.

Nota:

Un archivo CSS se guarda normalmente como **filename.css**.

Con CSS podemos modificar:

- color;
- tipografía;
- bordes;
- fondos;
- alineación;
- tamaño y distribución visual.

CSS, ejemplo conceptual:

Ejemplo conceptual (no es CSS real):

```
cubiertos {  
  color: gold;  
}
```



CSS, ejemplo real:

Ejemplo real:

```
p {  
  color: blue;  
  font-weight: bold;  
}
```

Significa:

“Todos los elementos `<p>` de la página deben ser azules y en negrita”.

Propiedades CSS básicas

Algunas propiedades frecuentes:

- `font-family`
- `color`
- `background-color`
- `border`
- `text-align`
- `width`

También es importante recordar que el comportamiento visual de un elemento depende de su tipo de visualización.

Vincular CSS en HTML

Plantilla básica. En el bloque head se enlace al archivo .css:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Ingeniería web</title>
    <link rel="stylesheet" href="filename.css" />
  </head>

  <body>
    ... contenido de la página ...
  </body>
</html>
```

CSS inline

El CSS inline (en línea) se aplica directamente dentro de una etiqueta HTML utilizando el atributo style

```
<p style="color: blue; font-size: 16px;">Este es un texto con CSS inline.</p>
```

Es ideal para cambios rápidos o correcciones, pero no se recomienda para proyectos grandes debido a problemas de mantenimiento, reutilización y saturación del código.

Pregunta: ¿Que forma de referencia CSS tiene mayor prioridad?

Algunas propiedades CSS

Ejemplos:

- Fuente: `font-family: Helvetica;`
- Color del texto: `color: gray;`
- Color de fondo: `background-color: red;`
- Borde: `border: 3px solid green;`
- Alineación del texto: `text-align: center;`

MDN (Mozilla Developer Network) es una de las mejores referencias para HTML y CSS.

Formas principales de definir colores en CSS

- Nombres predefinidos:

- `color: black;`

- Hexadecimal:

- `color: #00ff00;`

- `color: #0f0;`

- `color: #00ff0080;`

- `rgb()` y `rgba()` :
 - `color: rgb(34, 12, 64);`
 - `color: rgba(0, 0, 0, 0.5);`

Notas:

- La **a** en `rgba` significa canal alfa o transparencia.
- En general se prefiere, de más descriptivo a menos descriptivo:
 - i. nombre predefinido
 - ii. `rgb` / `rgba`
 - iii. hexadecimal

Ejemplo 1: panel de avisos del curso

Primero construiremos un ejemplo sencillo con HTML y CSS para observar cómo se combinan estructura y estilo.

```
<h1>Panel del curso</h1>

<p>
  <strong>Avisos</strong><br />
  Práctica 1 disponible.<br />
  Entrega parcial: <span>lunes 22:00</span><br />
  Recurso recomendado:
  <a href="#">MDN Web Docs</a>
</p>
```

```
body {  
  font-family: Arial, sans-serif;  
}  
  
h1 {  
  text-align: center;  
}  
  
p {  
  border: 3px solid hotpink;  
  background-color: lavenderblush;  
}  
  
span {  
  background-color: #fff59d;  
}
```



http://localhost/avisos

Panel del curso

Avisos

- Práctica 1 disponible
- Entrega parcial: lunes 22:00
- Recurso recomendado: MDN Web Docs

www.curso-web.edu/recursos

Análisis del ejemplo 1

Observa lo siguiente:

- el título aparece separado del párrafo;
- el bloque rosado ocupa mucho ancho;
- el texto resaltado se puede envolver con un elemento pequeño;
- el enlace no siempre se comporta como esperamos al alinearlos directamente.

Estas "rarezas" se entienden cuando diferenciamos entre **block** e **inline**.

Tres comportamientos visuales clave

En términos prácticos, muchos elementos HTML se comportan como:

- **block**: ocupan el ancho disponible y fluyen de arriba hacia abajo;
- **inline**: ocupan solo el espacio necesario y fluyen de izquierda a derecha;
- **inline-block**: se ubican en línea, pero admiten ancho y alto.

Comprender esto evita gran parte de los errores de maquetación iniciales.

Elementos block

Un elemento **block**:

- normalmente ocupa todo el ancho disponible;
- puede tener ancho y alto;
- se coloca en una nueva línea;
- organiza el contenido de arriba hacia abajo.

Ejemplos comunes:

- `h1`
- `p`
- `div`
- `ul`
- `table`

Ejemplo 2: un título block ocupa todo el ancho

En este ejemplo el título es un elemento de bloque. Aunque el texto sea corto, su caja se extiende horizontalmente.

```
<h1>Bitácora del laboratorio</h1>  
<p>Hoy aprenderemos cómo se comportan los elementos de bloque.</p>
```

```
h1 {  
  border: 5px solid red;  
}
```



Bitácora del laboratorio

Hoy aprenderemos cómo se comportan los elementos de bloque.

El título ocupa todo el ancho disponible aunque su texto sea corto.

¿Qué aprendemos aquí?

El borde rojo revela la "caja" del elemento `h1`.

Aunque el contenido no llegue hasta el extremo derecho, el navegador reserva todo el ancho disponible porque `h1` es **block** por defecto.

Por eso muchos títulos aparecen "en su propia línea".

Ejemplo 3: modificar el ancho de un block

Como los elementos block sí aceptan ancho, podemos controlar su tamaño.

```
<h1>Bitácora del laboratorio</h1>  
<p>  
  El párrafo sigue debajo porque los elementos de bloque fluyen de arriba hacia  
  abajo.  
</p>
```

```
h1 {  
  border: 5px solid red;  
  width: 50%;  
}
```



Bitácora del laboratorio

El párrafo sigue debajo porque los elementos de bloque fluyen de arriba hacia abajo.

Ahora el `<h1>` usa solo la mitad del ancho.

Conclusión sobre block

Los elementos block:

- sí admiten `width` ;
- siguen apareciendo uno debajo del otro;
- resultan ideales para secciones, títulos, párrafos y contenedores grandes.

Ahora comparemos ese comportamiento con los elementos inline.

Elementos inline

Un elemento **inline**:

- ocupa solo el espacio que necesita su contenido;
- fluye junto al texto de izquierda a derecha;
- no usa ancho ni alto de la misma forma que un block;
- no inicia una nueva línea por sí solo.

Ejemplos comunes:

- `a`
- `strong`
- `em`
- `span`

Ejemplo 4: un inline ignora width

El siguiente código intenta asignar un ancho a `strong`, pero no tendrá el efecto esperado.

```
<strong>Guía oficial</strong>  
<a href="#">Aula virtual</a>  
<a href="#">Referencia CSS</a>  
<a href="#">Validador HTML</a>
```

```
strong {  
  border: 5px solid red;  
  width: 1000px;  
}
```



¿Qué ocurre en el ejemplo 4?

La propiedad `width` se ignora visualmente porque `strong` es un elemento **inline**.
El borde rojo se ajusta al texto contenido, no a un ancho grande definido por CSS.
Esta es una causa muy frecuente de confusión cuando una propiedad “no funciona”.

Elementos inline-block

El comportamiento **inline-block** combina dos ventajas:

- se coloca en línea, como un inline;
- acepta ancho y alto, como un block.

Es útil para:

- tarjetas;
- miniaturas;
- botones;
- elementos alineados horizontalmente con medidas definidas.

Ejemplo 5: tarjetas en línea con tamaño definido

```
<div class="etiqueta">HTML</div>  
<div class="etiqueta">CSS</div>  
<div class="etiqueta">JS</div>  
<div class="etiqueta">DOM</div>  
<div class="etiqueta">Eventos</div>
```

```
.etiqueta {  
  display: inline-block;  
  width: 110px;  
  height: 110px;  
  margin-right: 10px;  
  border: 2px solid gray;  
  border-radius: 12px;  
  text-align: center;  
}
```



¿Qué aprendemos del inline-block?

Cada tarjeta:

- tiene tamaño definido;
- conserva una forma de caja;
- sigue apareciendo al lado de las demás.

Ese comportamiento no sería posible del mismo modo con un inline puro.

La propiedad `display`

Con `display` podemos cambiar el tipo de visualización de un elemento.

Valores muy usados:

- `block`
- `inline`
- `inline-block`

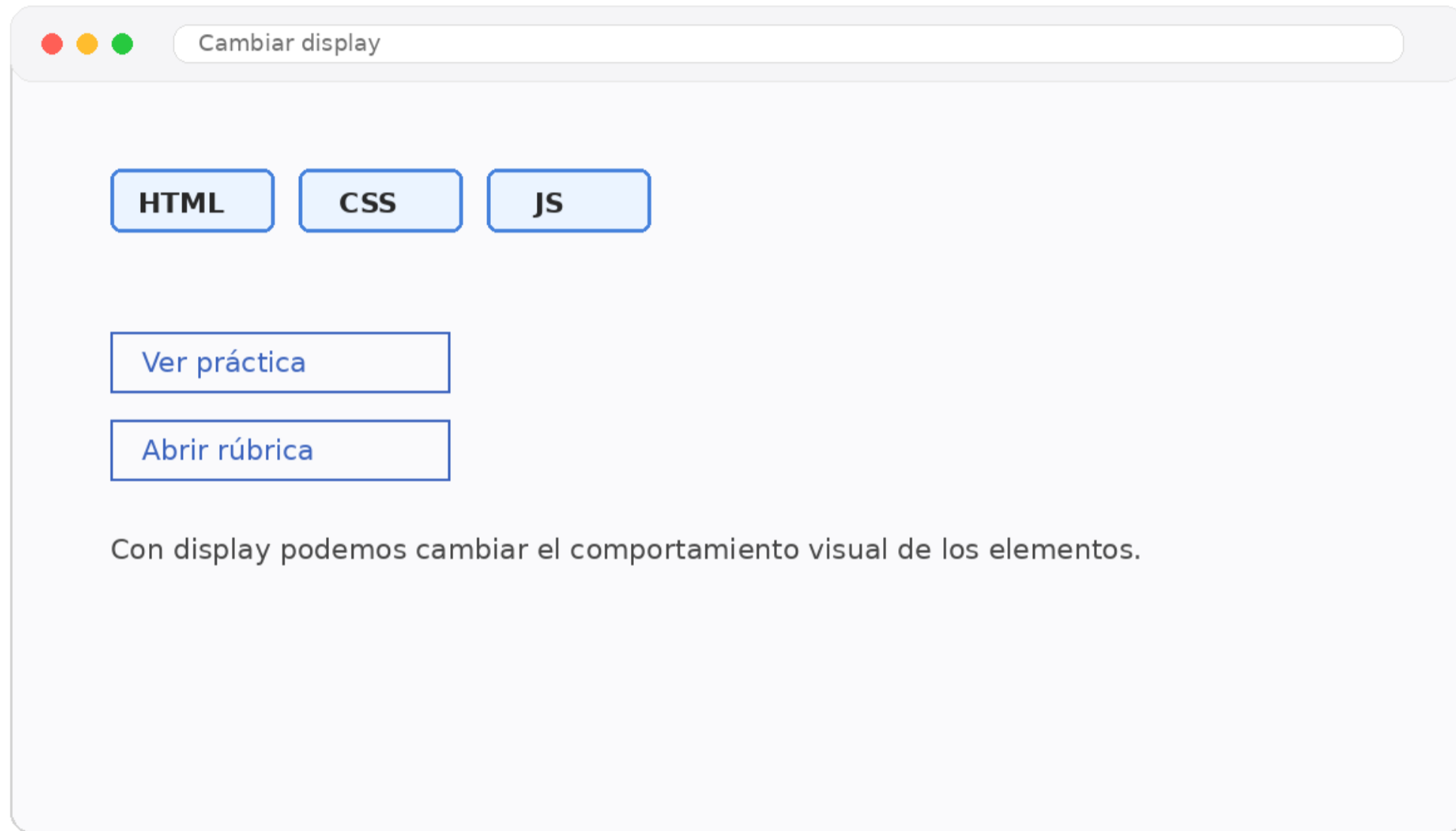
Esto permite adaptar la presentación visual sin cambiar la semántica del HTML.

Ejemplo 6: cambiar el comportamiento visual

```
<p>HTML</p>  
<p>CSS</p>  
<p>JS</p>
```

```
<a href="#">Ver práctica</a>  
<a href="#">Abrir rúbrica</a>
```

```
p {  
  display: inline;  
}  
  
a {  
  display: block;  
}
```



Lectura del ejemplo 6

Después del cambio:

- los párrafos se muestran uno al lado del otro;
- los enlaces pasan a ocupar líneas completas.

Esto demuestra que el comportamiento visual puede modificarse con CSS usando `display`.

Un problema común: resaltar solo una parte del texto

A veces no queremos modificar todo un párrafo, sino solo un fragmento.

Para eso usamos un contenedor inline genérico: `span`.

Si necesitamos un contenedor genérico de bloque, usamos `div`.

div y span

Dos elementos genéricos fundamentales:

- `div` : contenedor de bloque;
- `span` : contenedor en línea.

No aportan significado semántico por sí mismos, pero son muy útiles para:

- agrupar contenido;
- aplicar estilos específicos;
- organizar secciones visuales.

Clases e identificadores

Tres selectores básicos muy usados:

- selector por elemento: `p`
- selector por clase: `.pendiente`
- selector por id: `#titulo`

Regla práctica:

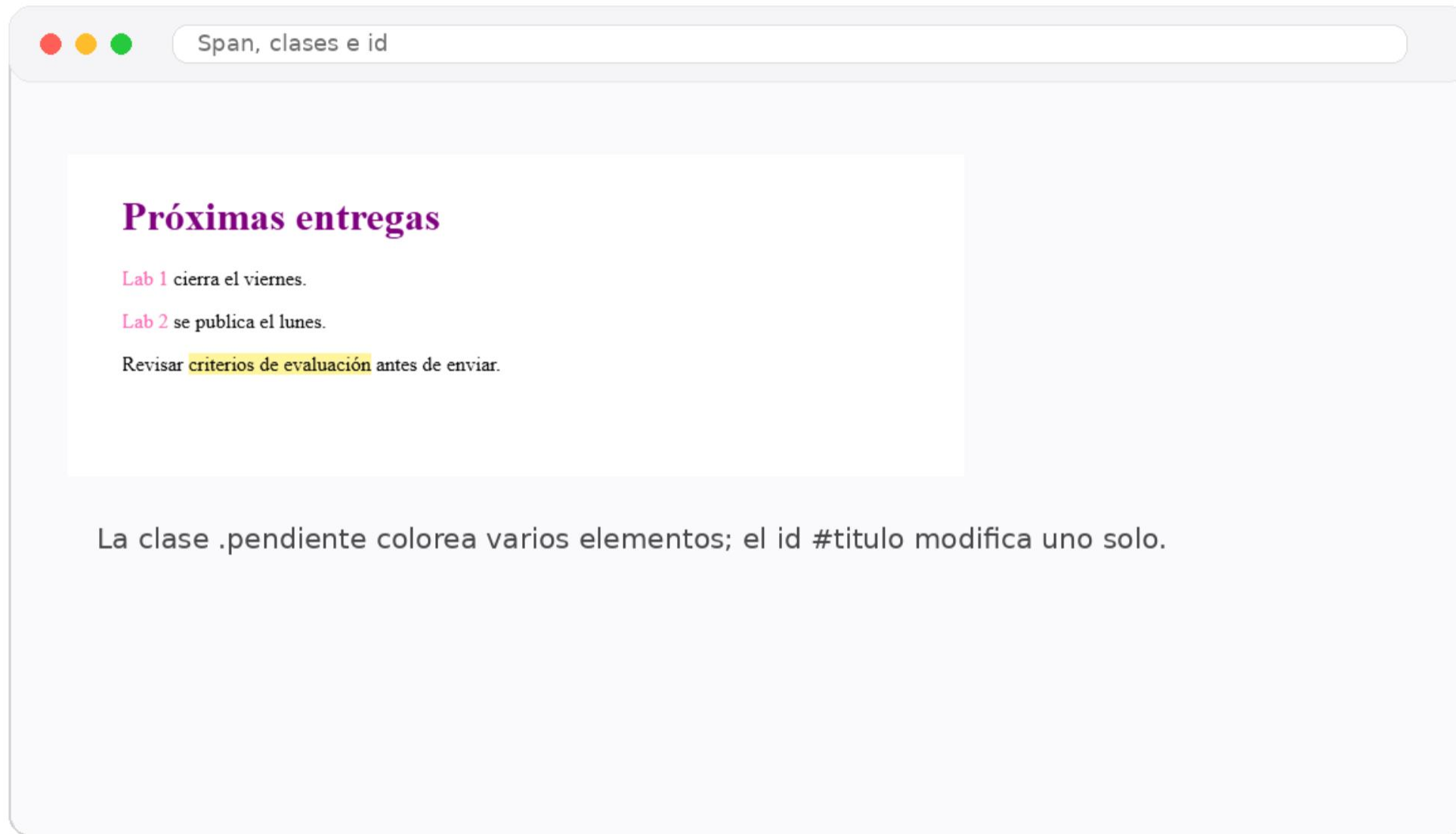
- una **clase** puede repetirse;
- un **id** identifica un único elemento dentro de la página.

Ejemplo 7: clases, id y resaltado con `span`

```
<h1 id="titulo">Próximas entregas</h1>

<p><span class="pendiente">Lab 1</span> cierra el viernes.</p>
<p><span class="pendiente">Lab 2</span> se publica el lunes.</p>
<p>
  Revisar <span class="resalte">criterios de evaluación</span> antes de enviar.
</p>
```

```
#titulo {  
  color: purple;  
}  
  
.pendiente {  
  color: hotpink;  
}  
  
.resalte {  
  background-color: #fff59d;  
}
```



¿Qué aporta este ejemplo?

Aquí se ve claramente que:

- `#titulo` afecta a un solo elemento;
- `.pendiente` puede reutilizarse en varios fragmentos;
- `span` permite intervenir solo una parte de una línea de texto.

Es una técnica básica, pero muy poderosa para construir interfaces.

Resumen conceptual

Ideas esenciales de la práctica:

- HTML estructura el contenido.
- CSS controla la presentación.
- Los elementos **block** ocupan ancho y van de arriba hacia abajo.
- Los elementos **inline** van en la misma línea y no manejan ancho igual que un block.
- Los elementos **inline-block** combinan flujo en línea con dimensiones.
- `display`, `class` e `id` ayudan a controlar mejor el diseño.

Actividad

Reproduce los ejemplos y luego modifica:

- colores;
- tamaños;
- bordes;
- alineación;
- tipo de `display` .

Después responde:

- ¿qué propiedades sí funcionan en cada tipo de elemento?
- ¿qué comportamiento cambió al usar `display` ?
- ¿cuándo conviene usar `span` y cuándo `div` ?

Actividad 2

Genera la siguiente landing page:

Jornada de Innovación Web 2026

Construyendo experiencias digitales accesibles y modernas

Participa en una jornada académica orientada a estudiantes de ingeniería, donde conocerás tendencias de desarrollo web, buenas prácticas de diseño y estrategias para crear interfaces más útiles y atractivas.

**WEB
LAB 2026**

Datos importantes

Fecha: 12 de mayo de 2026

Hora: 10:00 a. m.

Lugar: Auditorio de Ingeniería

Modalidad: Presencial con demostraciones en vivo

Temas principales

- HTML semántico para estructurar contenido
- CSS para mejorar presentación y legibilidad
- Buenas prácticas para interfaces accesibles
- Organización de contenido para sitios académicos

Programa del evento

1. Bienvenida e introducción
2. Demostración de una página web académica
3. Taller breve de maquetación
4. Preguntas y cierre

Registro: completa tu inscripción en línea para reservar tu participación.

[Registrarse ahora](#)

Vacantes limitadas para estudiantes de pregrado.

Jornada de Innovación Web 2026

Construyendo experiencias digitales accesibles y modernas

Participa en una jornada académica orientada a estudiantes de ingeniería, donde conocerás tendencias de desarrollo web, buenas prácticas de diseño y estrategias para crear interfaces más útiles y atractivas.

**WEB
LAB 2026**

Datos importantes

Fecha: 12 de mayo de 2026

Hora: 10:00 a. m.

Lugar: Auditorio de Ingeniería

Modalidad: Presencial con demostraciones en vivo

Temas principales

- HTML semántico para estructurar contenido
- CSS para mejorar presentación y legibilidad
- Buenas prácticas para interfaces accesibles
- Organización de contenido para sitios académicos

Programa del evento

1. Bienvenida e introducción
2. Demostración de una página web académica
3. Taller breve de maquetación
4. Preguntas y cierre

Registro: completa tu inscripción en línea para reservar tu participación.

Registrarse ahora

Vueltas limitadas para estudiantes de posgrado

Cierre

Dominar estas bases permite entender por qué una página se ve como se ve.

Antes de avanzar a maquetaciones más complejas, conviene practicar mucho con:

- cajas visibles;
- bordes;
- fondos;
- `display` ;
- selectores.

Ese hábito acelera el aprendizaje de CSS de manera notable.



¡Gracias! ➡